ThermalODE: HPC Simulator and Thermal-Aware Scheduler

Saket Sontakke, Taha Kapadia, Pranjal Vishwakarma, and Akshat Srivastava

***Abstract*—The increase in Artificial Intelligence (AI) and Deep Learning (DL) workloads has resulted in exponential growth in computational demands within High-Performance Computing (HPC) clusters. To satisfy these requirements, modern HPC cluster nodes are composed of dense, multi-GPU accelerated processors. However, this concentration of processors generates a significant amount of heat, elevating node temperatures to levels where the hardware is adversely affected or, in worse cases, enters a throttled state to prevent damage. During this throttled state, clock speeds are reduced, degrading the computational power of the processor and the overall efficiency of the cluster. This paper presents ThermalODE, an Ordinary Differential Equations (ODE) based physics-informed model trained on real-world telemetry from the MIT Supercloud Dataset to simulate the thermal behavior of individual nodes. The model was evaluated on over 443 million unseen timesteps across 671 jobs, and achieved a global Root Mean Square Error (RMSE) of 2.28°C and MAE of 1.45°C, validating its use as a digital twin for the cluster. This digital twin is then deployed to conduct simulations under 65% degradation in cooling efficiency using two scheduling algorithms: a standard First Come First Serve (FCFS) scheduler and a proposed thermal-aware scheduler that predicts the coolest available node prior to job allocation. Simulation of 56 Visual Geometry Group (VGG19) workloads across variable nodes (2 to 256 nodes) show that the thermal-aware scheduler took 21,149.61 seconds more waiting time on average than the standard scheduler. However, this trade-off resulted in better overall cooling. The thermal-aware scheduler reduced the global maximum temperature by 0.96°C and the average maximum temperature by 0.41°C on average across all node configurations as compared to the standard approach.**

***Index Terms*—High-Performance Computing (HPC), Ordinary Differential Equations (ODE), Physics-Informed Machine Learning, Digital Twin, Thermal Management.**

I. Introduction

THE exponential growth of artificial intelligence workloads, particularly large language models (LLMs) and deep learning frameworks, has increased the computational demand within HPC clusters substantially. To satisfy these requirements, modern architectures rely heavily on dense, multi-GPU accelerated server nodes. This high concentration of processors consume a large amount of power and as a result considerable heat is produced. Elevated temperatures not only accelerate hardware degradation, but also increase the energy costs associated with datacenter Heating, Ventilation, and Air Conditioning (HVAC) systems.

Traditional HPC workload managers like SLURM [1] allocate resources mainly on the availability of the nodes, job priority, and number of requested cores. However, the thermal condition of GPUs to which the tasks are to be assigned is unknown to the scheduler. As a result when multiple intensive tasks exhaust the GPUs to reach a threshold e.g. 87.0°C on NVIDIA Volta architectures [2], then the processor's clock speed is reduced, this event is known as throttling. Throttling saves the hardware from being damaged but increases the execution time taken by the task to be completed. GPUs also have a critical shutdown threshold e.g. 90.0°C on NVIDIA Volta architectures at which point the hardware triggers an emergency shutdown to prevent permanent silicon damage.

In order to solve such challenges a scheduler must be able anticipate thermal conditions before jobs are allocated to the GPU, rather than reacting after throttling has already occurred. A model that can accurately predict thermal behaviour of GPUs is crucial. Traditional Computational Fluid Dynamics (CFD) models are too computationally expensive and time consuming for real-time scheduling decisions. Pure data-driven models tend to overfit, they do not adhere to physical constraints and thus can predict values that violate the laws of thermodynamics. A physics-informed ODEs provides a middle ground as they are lightweight enough for real-time use and heat balance equations govern the ODEs which ensures that the predicted results remain within physical bounds.

This paper presents ThermalODE, a physics-informed digital twin and proactive thermal-aware scheduler for HPC clusters. The primary contributions of this work are as follows: The primary contributions of this work are as follows:

- **Thermodynamic Prior Extraction:** Empirical calculation of real-world lumped-parameter thermal priors thermal resistance, convective cooling coefficients, and cross-GPU crosstalk fractions directly from production NVIDIA V100 hardware telemetry of the MIT TX-Gaia cluster, providing reusable physics-grounded constants for future HPC thermal modeling work.
- **Physics-Informed ODEs:** A Two-Mass ODE model trained using the PyTorch deep learning library [3] via bounded gradient descent that achieves a global test RMSE of 2.28°C across over 443 million unseen timesteps, validating its deployment as an accurate digital twin of the MIT TX-Gaia cluster.
- **Thermal-Aware Scheduling Simulator:** A cluster simulator built using Numba Just-in-time (JIT) compiler [4] on the calibrated ODEs, capable of simulating standard FCFS (thermally blind) scheduler, a proposed thermal-aware scheduler or both using A/B testing feature.
- **Interactive Web Dashboard:** A visualization web application interface for telemetry replay and scheduling

The Authors are with the Department of Computer Science and Engineering, Dr. Vishwanath Karad MIT World Peace University, Pune, Maharashtra, India (e-mail: sontakkesaket9@gmail.com, taha.kapadia101@gmail.com, pranjal.vishwakarma1711@gmail.com, akshatsrivastava1975@gmail.com)

policy analysis in real-time, making the framework accessible without dedicated HPC infrastructure.

The paper is organized as follows. Section II reviews related work in HPC scheduling and datacenter thermal modelling. Section III details the data acquisition and data preprocessing pipeline. Section IV outlines the extraction of baseline hardware thermodynamic priors. Section VI presents the architecture and calibration of the ThermalODE model. Section VII describes the JIT compiled simulator and the logic for implemented scheduling strategies. Section VIII explains the web-based interactive visualization application. Finally, Section IX evaluates the performance of the model and discusses the trade-offs of the scheduling techniques, followed by concluding remarks in Section X.

II. Related Work

With increased computing power comes higher thermal density, which is becoming a major challenge for resource managers in High-Performance Computing (HPC). Traditional, "thermally blind" resource managers have become ineffective especially under conditions of long-running intensive jobs. In our analysis of the recent scientific literature, we distinguish three directions where the field has seen progress: reactive scheduling, model-based prediction of temperature, and digital twin-based orchestration.

A. Reactive Scheduling and Hardware Throttling

Traditional cluster orchestration is based on resource utilization rates and job priority rules without taking into account the actual thermodynamic state of the compute nodes. Recent empirical studies revealed that thermal hotspots along with Dynamic Voltage and Frequency Scaling (DVFS) were key causes behind execution skews and reduced throughput in multi-GPU systems [5]. Initial research into solving this problem involved only node-level optimization, such as thermal-aware scheduling frameworks that demonstrated the ability to cool down processors between 6°C to 12.2°C in single-node integrated CPU-GPU systems [6]. However, implementing similar heuristics on the scale of distributed HPC clusters is highly complicated due to the need for harsh power capping that affects makespan efficiency of the system [7].

B. Thermal Prediction Using Data Analysis

In shifting toward proactive control, the HPC domain has extensively focused on using machine learning (ML) to predict thermal conditions. Machine learning (ML) prediction techniques and neural networks have proved their effectiveness in predicting short-term temperature rises [8]. Extending these studies, deep reinforcement learning (RL) methods have been applied to schedule computational resources in clusters to prevent thermal emergencies by placing workloads intelligently and managing cooling systems [9]. Purely data-driven approaches exhibit great precision when dealing with scenarios similar to those used in training but are not physically meaningful and tend to break down in extreme scenarios outside the training distribution range, which has led to exploration of physically-consistent networks [10].

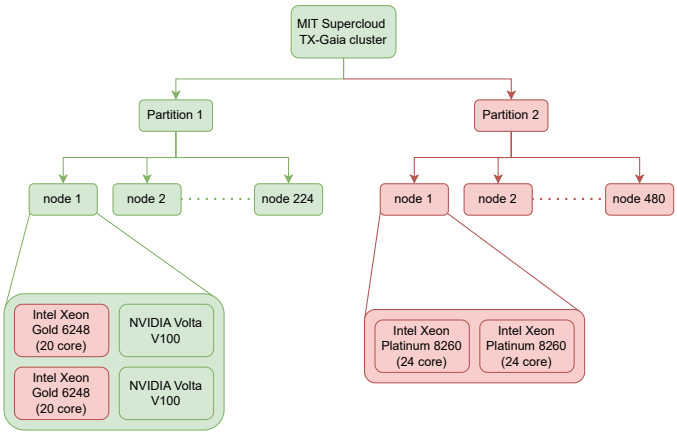

Fig. 1. Structural hierarchy of the MIT Supercloud TX-Gaia cluster.

C. Digital Twins and Predictive Control

In the present state-of-the-art, digital twins are utilized to simulate and assess scheduling schemes before their execution. Large-scale supercomputing centers stressed the importance of using digital twins for the evaluation of the intricate trade-off between job scheduling policies and power and cooling incentives [11]. Environments like DataCenterGym and SustainGym provide macro-scale simulations grounded on physical processes that include HVAC systems and the consideration of carbon footprint. On the other hand, systems like TAPAS [12] and SchedTwin [13] use digital twins to simulate at runtime the scheduling policies and dynamically adjust placement schemes, achieving notable recovery of throughput on inference workloads within warm datacenters.

III. Data Acquisition and Preprocessing

Predictive performance of ODE-based models relies greatly on the quality, diversity, scale and variance of the training data. Conversion of raw telemetry into clean, preprocessed tensors is therefore an integral part of this work.

A. Cluster Architecture and Scope

Telemetry data was sourced from the MIT Supercloud Dataset [14], which collected telemetry data the TX-Gaia heterogeneous cluster. The TX-Gaia system consists of two primary partitions. The first partition contains 224 nodes, each equipped with two 20-core Intel Xeon Gold 6248 processors (384GB RAM) and two NVIDIA Volta V100 GPUs (32GB RAM each). The second partition is entirely CPU-based, comprising 480 nodes with two 24-core Intel Xeon Platinum 8260 processors (192GB RAM) per node.

As illustrated in Fig. 1, this study is scoped exclusively to the NVIDIA Volta V100 GPUs within Partition 1 (green). Partition 2 was excluded entirely as it contains no GPU accelerators and is therefore irrelevant to GPU thermal modeling. Within Partition 1, the host Intel Xeon CPUs were also excluded for two reasons: fine-grained CPU temperature telemetry was not available in the dataset, and CPUs serve primarily as job dispatchers in GPU-accelerated HPC workloads rather than as the primary computational workhorses.

The V100 GPUs are where the overwhelming majority of compute, and by extension heat, is generated during AI and DL jobs, making them the sole relevant subject of thermal analysis.

B. Dataset Acquisition

The MIT Supercloud dataset provides a combination of high-level workload manager data and low-level job-specific Comma-Separated Values (CSV) time series data. GPU telemetry data, collected via the `nvidia-smi` utility at 100ms intervals, constitutes terabytes of time-series logs. To optimize development and computational overhead while maintaining a massive, representative sample, the preprocessing pipeline was initialized using 20% of the total available GPU subfolders (20 out of a total of 100 subfolders, specifically `gpu/0000` through `gpu/0019`). The dataset is publicly available and was acquired through the MIT Data Center Challenge repository[1].

C. Data Filtering and Cleaning

Acquired raw GPU time-series data consisted thousands of CSV files adding up to several hundred gigabytes in size. To reduce storage overhead and speed up downstream processing, all CSV files were first converted to Apache Parquet format using PyArrow. The resulting Parquet files were then scanned to identify nodes with consistent dual-GPU telemetry. A file was retained only if logs for both GPU 0 and GPU 1 were present, their row counts differed by no more than 5%, and their timestamp boundaries were within 5 seconds of each other. Files failing either condition were discarded, as misaligned GPU streams cannot be reliably paired for two-mass ODE training.

Filtered files were then aligned and cleaned. The asynchronous GPU 0 and GPU 1 telemetry streams were merged using a nearest-neighbor `merge_asof` with a 0.05-second tolerance window, and the unified timestamp was set to the midpoint of each matched pair. Absolute timestamps were then normalized to elapsed time from $t = 0$. Three additional filters were applied: files missing required columns (temperature, power draw, GPU and memory utilization) were dropped; files with fewer than 50 paired rows were discarded as too short for meaningful ODE calibration; and files with no thermodynamic excitation (defined as no single timestep with a temperature delta $\geq 1.0°$C or power delta ≥ 5.0W across either GPU) were removed, as static idle traces carry no useful thermal dynamics for training.

D. Stratified Splitting and Tensor Generation

To ensure that ThermalODE generalized across a diverse range of operational conditions, a global 2D stratification strategy was employed prior to the dataset split. Table I illustrates the binning across two dimensions: temporal length (Short, Medium, Long) and thermodynamic density (Low, Medium, High). A deterministic 80%, 10% and %10 split was then applied within each stratum which resulted in 5,299 jobs in training set, 659 jobs in validation set of, and 671 jobs in test set, as detailed in Table II.

[1]https://dcc.mit.edu/data/

TABLE I
2D STRATIFICATION MATRIX OF PHYSICAL JOB TRAJECTORIES

Length / Density	Low	Medium	High	**Total**
Short	938	1,028	244	2,210
Medium	401	715	1,093	2,209
Long	871	466	873	2,210
Total	2,210	2,209	2,210	**6,629**

TABLE II
DETAILED 80-10-10 SPLITTING FOR STRATA TRAJECTORIES

Strata Name	**Total**	**Train (80%)**	**Val (10%)**	**Test (10%)**
Short_Low	938	750	93	95
Short_Medium	1,028	822	102	104
Short_High	244	195	24	25
Medium_Low	401	320	40	41
Medium_Medium	715	572	71	72
Medium_High	1,093	874	109	110
Long_Low	871	696	87	88
Long_Medium	466	372	46	48
Long_High	873	698	87	88
Grand Total	**6,629**	**5,299**	**659**	**671**

Furthermore, the ambient temperature T_{amb} was estimated per job, as it is required by the ODE as an initial boundary condition but is not directly recorded in the available time-series data. The first 100 rows of each job trace were scanned for an idle period where both GPU utilizations were zero. If found, the cooler of the two GPUs at that timestep was used to back-calculate T_{amb} by inverting the steady-state heat transfer equation:

$$T_{\text{amb}} = T_{\text{die}} - \frac{P_{\text{self}}}{h} \tag{1}$$

where h is the empirically derived convective cooling prior (explained in Section IV) for that GPU. If no idle period existed (a hot-start job), the first row was used as the fallback. This approach avoids the need for a dedicated ambient sensor in the telemetry stream.

The resulting split datasets consisted of 1,705,242,321, 196,157,875, and 221,907,690 timesteps in training, validation and testing set respectively. These were then combined into fixed-length segments of 5,000 timesteps each which resulted in 343,503 training segments, 39,533 validation segments, and 44,691 test segments. Shorter terminal segments were zero-padded to maintain uniform tensor dimensions. A branching stack strategy was implemented to prevent RAM exhaustion during training:

- **Training Data:** 100,000 segments were stacked into a single PyTorch `.pt` chunk file, producing a total of 4 chunk files across the 343,503 training segments.
- **Validation Data:** All 39,533 segments were packed into a single tensor file for high-speed evaluation after every epoch.

- **Test Data:** Preserved with a strict 1-to-1 physical mapping (one `.pt` file per original Parquet job) across 671 test jobs, guaranteeing evaluation over contiguous, uncorrupted physical trajectories.

IV. Extraction of Thermodynamic Priors

A physics-informed ODE cannot be trained the same way as a conventional black-box neural network. Rather than allowing gradient descent to search an unbounded parameter space, the physical constants governing heat transfer such as thermal capacitance, resistance, and cooling coefficients must be constrained to values that are thermodynamically feasible for the specific hardware being modelled. These constraints are referred to as thermodynamic priors. These priors are empirically derived estimates of the true physical constants of the NVIDIA V100 GPUs in the MIT TX-Gaia cluster, calculated directly from the telemetry data before any model training begins.

Providing these priors as accurately as possible achieves two purposes. Firstly, they initialize the ODE parameters close to their true physical values, which gives the gradient descent a reasonable starting point rather than a random arbitrary one. Secondly, lower and upper bounds can be applied on these parameters, preventing the model from learning physically impossible behaviour such as negative thermal mass or heat flowing from cold to hot.

The cleaned dataset consisted of 6,629 dual-GPU job files and these were analyzed in two stages to extract the priors:

1) Calculation of thermal crosstalk coefficients (k), the heat transfer from one GPU to its neighbouring GPU residing in the same node.
2) Calculation of lumped-parameter physical constants, namely thermal capacitance (C), thermal resistance (R_{th}), convective cooling coefficient (h), and the dimensionless crosstalk prior (κ).

A. Thermal Crosstalk Calculation

In cluster nodes containing dual-GPU hardware, adjacent GPUs share a local thermal environment, resulting in passive heat transfer between the GPUs residing within the same node. To accurately capture these node level thermal dynamics the dataset needs to be standardized. The dual-GPU data that was obtained after data cleaning was re-sampled into uniform intervals of one second. Missing values were filled in with estimates using linear interpolation, ensuring a continuous and synchronized time-series for thermal crosstalk calculation. To isolate pure thermal crosstalk and eliminate multicollinear noise from concurrent workloads, the dataset was strictly scanned to locate *Asymmetric Events*.

An asymmetric event was defined as a continuous block of at least 30 seconds in which one GPU was subjected to a sustained heavy workload (power draw $P_{\text{active}} \geq 150.0\,\text{W}$) while the adjacent GPU remained entirely idle (power draw $P_{\text{idle}} \leq 50.0\,\text{W}$). An event was retained only if the idle GPU exhibited a passive temperature rise of $\Delta T \geq 1.0\,°\text{C}$ above its baseline temperature at event onset. For each valid event, the directional thermal crosstalk coefficient was computed as:

TABLE III
Thermal Priors for GPU 0 and GPU 1

Parameter	Unit	GPU 0	GPU 1
Capacitance Prior (C)	$J/°C$	143.22	140.25
Resistance Prior (R_{th})	$°C/W$	0.2053	0.1856
Convective Cooling Prior (h)	$W/°C$	4.8713	5.3871
Crosstalk Prior (κ)	Dimensionless	0.0780	0.0281

$$k = \frac{\Delta T}{\bar{P}_{\text{active}}} \tag{2}$$

where $\bar{P}_{\text{active}}$ is the mean power draw of the active GPU over the event window.

Across 6,628 successfully processed files, 1,270 asymmetric events isolating GPU 1 heating GPU 0 were identified, yielding a mean crosstalk coefficient $k_{01} = 0.01602\,°\text{C/W}$. A total of 93 events isolating GPU 0 heating GPU 1 were identified, yielding $k_{10} = 0.00522\,°\text{C/W}$.

B. Lumped Parameter Thermal Prior Extraction

Following the establishment of directional crosstalk coefficients, the intrinsic thermodynamic parameters of each GPU were extracted. The cleaned dataset of 6,629 dual-GPU job files was scanned for step-response heating curves, defined as intervals initiated by a rapid power spike exceeding 100.0 W, sustained for at least 30 seconds, and captured within a maximum observation window of 150 seconds. Files where power dropped below 50% of the spike threshold before 30 seconds elapsed were discarded as insufficiently sustained.

For each identified heating phase, the telemetry was first resampled to uniform 1-second intervals via linear interpolation. An exponential step-response curve of the form:

$$T(t) = T_{\text{idle}} + T_{\text{amp}}\left(1 - e^{-t/\tau}\right) \tag{3}$$

was fitted to the heating trace using `scipy.optimize.curve_fit` with bounded constraints: $T_{\text{amp}} \in [3.0, 100.0]\,°\text{C}$ and $\tau \in [5.0, 400.0]\,\text{s}$. Curves where the fitted τ collided with either boundary were rejected as physically implausible. From the accepted fits, the lumped thermal resistance and convective cooling coefficient were derived as:

$$R_{th} = \frac{T_{\text{amp}}}{P_{\text{heat}} - P_{\text{idle}}}, \quad h = \frac{1}{R_{th}}, \quad C = \frac{\tau}{R_{th}} \tag{4}$$

with a final sanity check requiring $C \in [20.0, 1000.0]$ J/°C. The previously calculated thermal crosstalk coefficients ($k_{01} = 0.01602\,°\text{C/W}$ and $k_{10} = 0.00522\,°\text{C/W}$) are converted to dimensionless crosstalk priors using:

$$\kappa_{ij} = k_{ij} \cdot h_i \tag{5}$$

Of the 6,629 scanned files, 2,453 yielded valid step-response curves 1,624 contributing fits for GPU 0 and 2,148 for GPU 1. The remaining 4,176 files were discarded due to insufficient thermal excitation or curve-fit rejection. The median priors extracted across valid fits are summarized in Table III.

The close agreement between C_0 and C_1 (less than 2.1% relative difference) supports the assumption of similar thermal mass across the two GPUs, consistent with their identical hardware specification. From the observed priors and the divergence in h values it can be inferred that airflow conditions within the node are asymmetric. GPU 1 likely sits upstream in the airflow path, meaning its thermal exhaust is carried directly over GPU 0, which explains the higher crosstalk coefficient ($k_{01} = 0.01602\,°\text{C/W}$) and higher thermal resistance ($R_{th,0} = 0.2053\,°\text{C/W}$) observed for GPU 0. Conversely, heat generated by GPU 0 must propagate against the primary airflow to reach GPU 1, resulting in a significantly lower crosstalk coefficient ($k_{10} = 0.00522\,°\text{C/W}$) and lower thermal resistance ($R_{th,1} = 0.1856\,°\text{C/W}$) for the upstream GPU.

V. Model Architecture Evolution

The final physics-informed ODE architecture evolved through an iterative, data-driven process rather than starting from a single, pre-planned design. The development of the three model iterations was guided by actively monitoring the training logs, with execution halted whenever a model hit mathematical or physical limits. As a result, the first two iterations were manually stopped before reaching their planned epoch budgets. Their performance metrics reflect mid-development snapshots rather than fully converged states; only Iteration 3 was trained to complete convergence. Because of this difference in training budgets, a direct numerical comparison of their final performance is not meaningful and is therefore omitted.

To manage the expanding complexity across these iterations, we used the empirically derived values from Section IV as starting points where applicable. However, as the architecture evolved to capture non-linear and multi-mass dynamics, our reliance on these simple step-response priors shifted. Newly introduced parameters were initialized using standard engineering heuristics. Throughout all iterations, the parameters were mathematically restricted to remain within realistic physical limits during training, ensuring the optimizer could not learn impossible behaviours like negative thermal mass. The exact parameter initializations and boundary limits for the final architecture are provided in Section VI.

A. Iteration 1: Single-Mass Static Cooling

The first iteration modelled the entire GPU thermal assembly as a single lumped capacitance C with a static linear cooling coefficient h. For this iteration, the empirically derived priors (C, h, and κ) were used directly as the initial weights. Training was manually stopped at epoch 35 out of a planned 50 after the logs showed signs of early stagnation. The validation RMSE plateaued around 6.16 °C, with improvements dropping below 0.002 °C per epoch. Since the optimizer was stuck in a local minimum and a high error, the training was paused to investigate the bottleneck. Analyzing the residuals at the epoch 35 checkpoint showed that the model consistently underpredicted temperatures above 70 °C, completely failing to track high-temperature plateaus. This made physical sense: a static h parameter represents a constant cooling rate, meaning it mathematically cannot capture the aggressive, non-linear way GPU fan curves engage at higher temperatures. This limitation prompted the move to Iteration 2.

B. Iteration 2: Single-Mass Dynamic Cooling

To address the fan curve issue, a sigmoid-gated, temperature dependent cooling function $h(T)$ was introduced. While the empirical priors for capacitance (C) and crosstalk (κ) were retained, the static h was replaced by new dynamic cooling parameters (h_{base} and h_{active}), which were initialized using engineering heuristics. This second iteration was set to run for 500 epochs but was manually halted early at epoch 195. Although the validation RMSE initially improved, the error trajectory eventually plateaued with an improvement of only 0.02°C over the last 50 epochs. More importantly, the parameter evolution showed the lumped capacitance C drifting continuously upward until it hit 993.81 J/°C, saturating against our predefined physical upper bound of 1000.0 J/°C. Essentially, the optimizer was trying to inflate the thermal mass to artificially dampen errors it could not physically fit. Due to the error plateau and parameter saturation, the training was stopped. Evaluating the epoch 195 checkpoint revealed persistent tracking errors during sudden power spikes. This lead to an understanding that governing the temperature differential $\frac{dT}{dt}$ with a single lumped capacitance C simply cannot capture both the near-instantaneous heating of the die and the slow thermal soaking of the massive heatsink. This proved that the model needed to be structurally decomposed.

C. Iteration 3: Two-Mass Dynamic Cooling

Guided by previous failure modes, the lumped thermal mass was split into two distinct masses: a silicon die (C_{die}) and a heatsink (C_{sink}), coupled by thermal paste resistance (R_{paste}). Because the single empirical C value could not accurately represent this decoupled system, it was discarded and heuristic starting points were used for both masses. Ultimately, only the empirical crosstalk prior (κ) was carried forward directly from Section IV. This model was trained to convergence and the parameters quickly stabilized within realistic physical boundaries. Early stopping identified the optimal weights at epoch 81, achieving a minimum validation RMSE of 2.28 °C. It successfully captured both the high-temperature plateaus and the fast power spikes. This two-mass architecture with dynamic cooling was finally adopted for all subsequent analysis.

VI. The Physics-Informed ODE Model

Following the iterative development process detailed in Section V, the Two-Mass Dynamic Cooling architecture was selected as the optimal framework for representing the thermal behavior of the dual-GPU nodes. This section provides the comprehensive mathematical formulation of this finalized model.

A. Two-Mass ODE Architecture

The physical hardware was developed as a Two-Mass model for each GPU, splitting the thermal assembly into two masses:

the Silicon Die (C_{die}), where heat is generated, and the Heatsink mass (C_{sink}), where heat is dissipated. Heat flows from the die to the heatsink through the thermal interface material, represented by thermal paste resistance (R_{paste}). The heatsink subsequently dissipates energy into the ambient environment (T_{amb}) via convective cooling (h). Since T_{amb} is not directly measured in the telemetry, it is estimated per job during preprocessing as described in Section III.

The heatsink temperature at $t = 0$ is not directly observable in the available timeseries logs. It is therefore initialized at the start of each sequence by assuming initial thermal equilibrium through the paste:

$$T_{\text{sink}}(0) = T_{\text{die}}(0) - P(0) \cdot R_{\text{paste}} \tag{6}$$

For a given GPU, the continuous-time thermal dynamics are governed by the following system of differential equations:

$$\frac{dT_{\text{die}}}{dt} = \frac{1}{C_{\text{die}}}\left(P - \frac{T_{\text{die}} - T_{\text{sink}}}{R_{\text{paste}}}\right) \tag{7}$$

$$\frac{dT_{\text{sink}}}{dt} = \frac{1}{C_{\text{sink}}}\left(\frac{T_{\text{die}} - T_{\text{sink}}}{R_{\text{paste}}} + k \cdot P_{\text{adj}} - h \cdot (T_{\text{sink}} - T_{\text{amb}}) + q\right) \tag{8}$$

Equation (7) describes the heat balance directly at the silicon level. As the GPU processes workloads, the die generates heat proportional to its power draw (P_{self}). Because the die possesses very little physical mass, its thermal capacitance (C_{die}) is extremely small, meaning its temperature can spike in a matter of seconds. The only way this heat can escape is through conduction across the the thermal paste into the heatsink. This transfer rate is defined by the temperature gradient between the die and the sink, but slowed down by the paste's thermal resistance (R_{paste}).

Equation (8) characterizes the behavior of the heatsink. Unlike the die, the heatsink is massive, and its large thermal capacitance (C_{sink}) acts as a thermal reservoir that heats up significantly slower than the silicon die. It not only accumulates heat flowing in from the die, but also absorbs thermal crosstalk ($k \cdot P_{\text{adj}}$) radiating from the adjacent GPU's power draw. To cool down, the heatsink dumps energy into the server's passing airflow. This heat convectional dissipation is driven by the difference between the sink temperature and the ambient intake air (T_{amb}), and is scaled by the dynamic fan cooling rate $h(T_{\text{die}})$. Finally, a static bias term (q) is added to compensate for localized background heat from unmodeled components, such as nearby CPUs or neighbouring nodes.

The physical grounding of both GPUs within a node is identical but the training process maintains fully independent parameter sets for GPU 0 and GPU 1. As noted in the Section IV, upstream and downstream GPUs experience vastly different airflows and calibrating the parameters for both GPUs separately allows the model to inherently obey these physical asymmetries. A complete breakdown of all the 18 trainable parameters, including their description, physical units, the parameter boundaries and the final optimized values are provided in Table IV.

Although equations (7) and (8) define the continuous thermal physics of the GPUs, the actual MIT Supercloud telemetry is discrete, sampled approximately every 100 milliseconds. To map these continuous equations into deep learning frameworks such as PyTorch and evaluate them against the data, the Forward Euler method is used to numerically integrate the ODEs. A fixed integration timestep of $\Delta t = 0.11$ seconds is used, which closely aligns with the 100ms logging interval while also accounting for minor sensor latencies. Instead of attempting to solve for a continuous curve, the model steps through the sequence chronologically. At each timestep, it evaluates the current temperature gradients and uses them to project the die and heatsink temperatures forward into the next discrete state:

$$T_{\text{die}}(t + \Delta t) = T_{\text{die}}(t) + \Delta t \cdot \frac{dT_{\text{die}}}{dt} \tag{9}$$

$$T_{\text{sink}}(t + \Delta t) = T_{\text{sink}}(t) + \Delta t \cdot \frac{dT_{\text{sink}}}{dt} \tag{10}$$

By propagating these iterative updates across the entire sequence length of the input tensors, the model generates a complete thermal trajectory. However, computing the heatsink gradient at any given timestep requires knowing the exact convective cooling rate at that precise moment. This requires a mathematical representation of the hardware's variable fan speeds, detailed below.

B. Dynamic Active Cooling Modeling

Datacenter GPUs utilize dynamic fan curves that non-linearly increase the convective cooling rate when die temperatures exceed a hardware defined threshold. To capture this behavior while maintaining differentiability for backpropagation, the cooling coefficient h is modeled as a sigmoid-gated function of the current die temperature:

$$h(T_{\text{die}}) = h_{\text{base}} + h_{\text{active}} \cdot \sigma\Big(\beta(T_{\text{die}} - T_{\text{thresh}})\Big) \tag{11}$$

where h_{base} represents passive baseline airflow at idle fan speeds, h_{active} represents the maximum additional cooling capacity at full fan duty cycle, T_{thresh} is the die temperature at which the BIOS begins spinning up the fans, and β controls the steepness of the fan engagement curve. The sigmoid operator σ is a fixed mathematical function and carries no calibrated value of its own.

C. Physics-Informed Parameter Bounding

The model has 18 independent trainable parameters across the two GPUs. Unconstrained gradient descent is prone to converging on physically impossible solutions, such as negative thermal mass or heat flowing from cold to hot. To strictly enforce thermodynamic boundaries, all trainable parameters are stored in an unbounded raw space and projected back into their physical ranges during each forward pass using a sigmoid mapping:

TABLE IV
ODE TRAINABLE PARAMETERS DESCRIPTION AND VALUES

Symbol	Description	Unit	Init	Lower Bound	Upper Bound	Optimized (GPU 0)	Optimized (GPU 1)
C_{die}	Silicon die heat capacity	J/°C	2.0	0.1	10.0	8.9324	8.8707
C_{sink}	Heatsink thermal mass	J/°C	500.0	100.0	5000.0	4713.5889	4831.1543
R_{paste}	Thermal paste resistance	°C/W	0.05	0.001	0.2	0.0366	0.0336
h_{base}	Passive cooling coeff.	W/°C	4.0	0.5	10.0	3.9791	4.7617
h_{active}	Max active cooling	W/°C	12.0	0.0	30.0	20.9114	19.8178
T_{thresh}	Fan activation temp.	°C	65.0	40.0	85.0	70.2507	67.5134
β	Fan curve steepness	1/°C	0.5	0.05	2.0	1.6688	1.3351
k_{01}	Crosstalk GPU1→GPU0	unitless	0.0780	0.0	0.5	0.0167	—
k_{10}	Crosstalk GPU0→GPU1	unitless	0.0281	0.0	0.5	—	0.0028
q	Ambient heat bias	W	0.0	-10.0	10.0	-8.9202	-8.9428

$$p = p_{\text{low}} + \sigma(\hat{p}) \cdot (p_{\text{high}} - p_{\text{low}}) \tag{12}$$

where $\hat{p}$ is the raw unconstrained `nn.Parameter` value optimized by Adam [15], and p_{low}, p_{high} are the physical bounds. The parameters are initialized by inverting this mapping from their prior values:

$$\hat{p}_{\text{init}} = \sigma^{-1}\left(\frac{p_{\text{init}} - p_{\text{low}}}{p_{\text{high}} - p_{\text{low}}}\right) = \ln\left(\frac{\text{norm}}{1 - \text{norm}}\right) \tag{13}$$

The description, units, initial values, physical bounds and final optimized values for all 18 parameters are summarized in Table IV.

D. Training Procedure

The model was set to train for up to 500 epochs utilizing the Adam optimizer. The learning rate is monitored by a `ReduceLROnPlateau` scheduler and was set to 0.01 initially. If the validation RMSE shows no improvement for 3 consecutive epochs, the learning rate is halved.

Because the dataset segments vary in their valid sequence lengths, the loss function was defined as a masked Mean Squared Error (MSE). This is computed jointly across both modelled GPUs (GPU 0 and GPU 1) and strictly restricted to valid, non-padded timesteps to prevent zeroes from artificially skewing the loss gradient. The objective function is defined as:

$$\mathcal{L}(\theta) = \frac{E_{\text{error}}}{2\sum_{i=1}^{B}\sum_{t=0}^{S} M_{(i,t)}} \tag{14}$$

where E_{error} represents the sum of squared errors across all valid timesteps and modelled GPUs, defined as:

$$E_{\text{error}} = \sum_{i=1}^{B}\sum_{t=0}^{S}\Bigg[\Big((\hat{T}_0^{(i,t)} - T_0^{(i,t)})^2 + (\hat{T}_1^{(i,t)} - T_1^{(i,t)})^2\Big) \cdot M_{(i,t)}\Bigg] \tag{15}$$

where $\hat{T}_{\text{die}}$ is the predicted silicon die temperature, T_{true} is actual temperature from the dataset, $M_{(i,t)} \in \{0, 1\}$ is the boolean validity mask, B is the batch size, and S is the padded segment length of 5,000 timesteps. The RMSE is computed by taking the square root of the masked MSE $\mathcal{L}(\theta)$, returning an error in the same units as the temperature (e.g. °C), making it a more interpretable measure of average prediction deviation. RMSE is the metric used to saved the best model (least RMSE) during the training process and is defined as:

$$\text{RMSE} = \sqrt{\mathcal{L}(\theta)} \tag{16}$$

The physics-informed ODE model was trained on a workstation equipped with an NVIDIA RTX A4500 accelerator. To efficiently utilize this training hardware across the multi-gigabyte dataset, a custom `MultiChunkPrefetcher` was implemented. This daemon thread loaded training chunks from the solid-state drive (SSD) into pinned CPU RAM. To effectively mask I/O latencies, these chunks were streamed to the accelerator's VRAM via non-blocking CUDA streams.

To ensure robust generalization, a double-shuffle strategy was applied: training chunks were macro-shuffled prior to each epoch, and the sequences within each chunk were micro-shuffled directly on the compute accelerator. This on-device micro-shuffling eliminated severe CPU-to-accelerator synchronization overhead. Furthermore, to prevent repetitive and costly memory swaps, the full validation bundle of 39,533 segments was kept permanently resident in the RTX A4500's VRAM. During the evaluation phase at the end of each epoch, this validation set was processed in batches of 10,000 segments per forward pass to prevent out-of-memory (OOM) failures. Model checkpoints and parameter telemetry logs were saved per epoch, with the final model weights selected based on the strict minimum validation RMSE.

VII. HPC CLUSTER SIMULATOR WITH INTEGRATED SCHEDULERS

To evaluate the real-world efficacy of the calibrated physics-informed ODE, a high-performance JIT compiled digital twin cluster simulator was engineered. While the PyTorch training environment was optimal for backpropagation and parameter calibration, evaluating continuous scheduling policies across thousands of sequential jobs demanded execution speeds that interpreted Python could not provide. Consequently, both the two-mass ODE physics engine (`step_physics_numba`) and the candidate placement

logic (`find_best_placement_numba`) were decorated with Numba's `@njit(cache=True)` directive, compiling both functions to native machine code ahead of the simulation loop. A dedicated JIT warmup pass is executed once at startup to amortize compilation latency before any job is processed.

A. Job Ingestion and Trace Representation

The simulator ingests historical GPU power traces obtained from the MIT Supercloud dataset, stored as CSV or Parquet files. Each file is parsed into a job dictionary containing one or two power trace arrays (GPU 0 and GPU 1), along with Welford online [16] accumulators (`mean`, `M2`, `tick_count`, `min_temp`, `max_temp`) and a throttle-step counter per GPU. The Welford method was chosen deliberately to compute running mean and variance without retaining per-tick temperature histories in memory, which would become extensive for large node sweeps. On the first execution over a given input directory, all parsed jobs are serialized to a binary cache (`parsed_jobs_cache.pkl`), eliminating repeated I/O overhead on subsequent runs.

The simulator enforces realistic hardware protection throttling logic. When a GPU's die temperature reaches the throttle threshold $T_{\text{throttle}} = 87.0\,^{\circ}\text{C}$, the GPU transitions from `STATE_ACTIVE` to `STATE_THROTTLED` and its instantaneous power draw is hard-capped to $P_{\text{throttle}} = 100\,\text{W}$ via `THROTTLE_CAP`, regardless of the workload's actual demand. The GPU remains in this throttled state and the associated job's execution time is extended until passive convection recovers the die temperature below $T_{\text{recovery}} = 83.0\,^{\circ}\text{C}$. Should the die temperature reach the absolute shutdown threshold $T_{\text{shutdown}} = 90.0\,^{\circ}\text{C}$, the GPU transitions to `STATE_SHUTDOWN`, the job is marked as failed, and the node is freed to cool passively. Every elapsed tick during which a GPU is in `STATE_THROTTLED` is accumulated in the job's `throttledSteps` counter, from which a precise throttle duration in wall-clock seconds is derived at job completion.

The simulator supports three operating modes `STANDARD`, `THERMAL_AWARE`, and `AB_TESTING` selectable via the `Config.MODE` field. All physics constants and scheduler thresholds are centralized in `PHYSICS_PARAMS`, and the precomputed inverses of thermal capacitances and paste resistances are cached as module-level constants to avoid redundant division inside the JIT compiled kernel.

B. Standard Scheduler (FCFS)

To establish a thermally blind control, the Standard scheduler assigns each incoming job to the first available node with sufficient free GPU slots. The placement function scans `gpu_status` row by row, assembling a list of candidate `(node, gpu)` pairs, and unconditionally returns the first candidate (`cand_nodes[0]`) without consulting any thermal state. This faithfully replicates the first-fit behavior of conventional HPC resource allocators.

C. Thermal-Aware Scheduler (Predictive)

To preempt reactive throttling before it occurs, the Thermal-Aware scheduler modifies the GPU allocation step with a fast-forward thermal projection executed entirely within the JIT compiled kernel. Prior to any job dispatch, the scheduler copies the current die and heat-sink temperatures of all candidate nodes into a scratch array and runs a forward simulation of N_{proj} physics timesteps, applying a conservative worst-case power draw of $P_{\text{proj}} = 250\,\text{W}$ to all GPUs that the current job under consideration would occupy. Neighbouring GPUs on the same node that are already active are also held at P_{proj}, capturing inter-GPU heat transfer via the k_{01} and k_{10} crosstalk coefficients from the calibrated ODE. The projection horizon is set to five simulated minutes:

$$N_{\text{proj}} = \left\lfloor \frac{5 \times 60}{\Delta t} \right\rfloor = \left\lfloor \frac{300}{0.11} \right\rfloor = 2727 \text{ timesteps.} \tag{17}$$

After projecting all candidate GPUs in a single batched pass, the scheduler selects the candidate whose projected peak die temperature is minimal:

$$n^* = \arg\min_{i \in \mathcal{C}} \hat{T}^{(i)}_{\text{peak}}, \tag{18}$$

where $\mathcal{C}$ denotes the set of all valid candidate `(node, gpu)` pairs and $\hat{T}^{(i)}_{\text{peak}}$ is the maximum die temperature recorded across the projection horizon for candidate i. The winning candidate n^* then passes through two sequential rejection gates before the job is dispatched.

1) Gate 1: Projection Limit Rejection: If the winner's projected peak temperature exceeds $T_{\text{proj_limit}} = 70.0\,^{\circ}\text{C}$, the allocation is aborted and the job remains at the head of the queue. The scheduler exits the placement loop for the current tick and re-evaluates on the next simulation step, by which time the GPUs may have cooled down.

2) Gate 2: Cooldown Lock Enforcement: Even if the projection passes Gate 1, the scheduler checks the node's *current* die temperature at the moment of dispatch. If the current maximum die temperature of the candidate GPUs exceeds $T_{\text{safe}} = 65.0\,^{\circ}\text{C}$, the allocation is again rejected. This gate prevents immediately re-loading a node that has just completed a thermal-intensive workload but whose temperature has not yet recovered to a safe baseline, even when the projection horizon alone would appear acceptable.

Only a candidate that passes both gates proceeds to actual job assignment. By evaluating the thermodynamic future rather than the instantaneous thermal state, the Thermal-Aware scheduler eliminates hardware-induced throttling events, transferring latency from mid-execution throughput degradation to pre-execution queue wait time.

D. Output Artifacts and Archival

Upon completion of each simulation run, all output artifacts are organized and preserved systematically. Per-node telemetry is written to CSV files using a deferred, batched I/O strategy: numerical tuples $(t,\, T_{\text{die},0},\, T_{\text{die},1},\, P_0,\, P_1)$ are appended to in-memory buffers throughout the simulation and flushed to disk in a single `writelines` call after the main loop terminates, avoiding filesystem contention during the performance-critical inner loop. When `HIGH_RES_TELEMETRY` is disabled, telemetry is sampled at every 50th tick (approximately

5.5 s intervals) to reduce output volume; enabling it retains the full $\Delta t = 0.11$ s resolution. Interactive per-node HTML dashboards embedding Chart.js with pan-and-zoom capability are generated for post-hoc visual analysis of temperature and power over time. A per-run summary CSV records job-level statistics extracted from the Welford accumulators at completion, and a metadata JSON captures the system configuration, simulated makespan, and aggregate fleet-wide metrics. All of these artifacts are written to a temporary directory and then compressed into a single ZIP archive, after which the temporary directory is removed.

E. A/B Testing Mode

The `AB_TESTING` mode does not introduce any new scheduling logic. It is purely a convenience wrapper that executes the Standard scheduler and the Thermal-Aware scheduler back-to-back within a single invocation, feeding each an independent deep copy of the master job queue via `copy.deepcopy(master_queue)`. This isolation ensures that Welford accumulators and work-deficit state accumulated during the Standard run do not carry over into the Thermal-Aware run. Each scheduler produces its own full set of artifacts in an isolated subdirectory. In addition, the A/B run produces a merged side-by-side summary CSV, with Standard and Thermal-Aware per-job columns interleaved, and a combined metadata JSON recording the simulated makespan and aggregate thermal statistics for both policies. This makes a direct per-job comparison immediately available without any post-processing.

F. Cluster Scaling

The `Config.NODES` parameter accepts a non-empty Python list of integer node counts. Regardless of the selected `MODE`, the simulator iterates over this list sequentially, executing a complete simulation for each node count in turn. The run for a given tier must complete and its ZIP archive must be written to disk before the next tier begins. All physics constants, scheduler thresholds, the input trace directory, and the ambient temperature are specified once and held constant across every tier, so the node count is the sole independent variable. For example, setting:

```
NODES              = [32, 64, 128, 256]
AMBIENT_TEMP_C     = 30
MODE               = 'AB_TESTING'
HIGH_RES_TELEMETRY = False
```

causes the simulator to execute a full A/B pair at 32 nodes, archive the result, then proceed to 64 nodes, and so on through 256 nodes, all from a single execution. The same sequential sweep is equally valid with `MODE = 'STANDARD'` or `MODE = 'THERMAL_AWARE'`, in which case each tier produces a single-scheduler archive rather than a paired one. This design makes the full scaling study a fire-and-forget operation, the user configures the input once and obtains a structured set of per-tier archives suitable for direct comparative analysis across cluster scales.

VIII. Interactive Web-Based Visualization Dashboard

To provide accessible, real-time visualization User Interface (UI) of the simulator and the scheduler dynamics without requiring a local Python environment, an interactive browser-native digital twin was developed and deployed at https://thermal-ode.vercel.app. The implementation required a complete architectural transformation of the Python simulation stack into the browser runtime, preserving numerical accuracy while satisfying the strict concurrency and rendering constraints of the web platform.

A. TypeScript Physics Port

The two-mass ODE physics engine was fully ported from Python to TypeScript in `physics.ts`. The calibrated parameter set produced by the PyTorch training pipeline is stored as a static JSON file (`calibrated_physics.json`) and imported directly into the frontend module at build time. The combined physics parameter object is assembled at module load time by merging the JSON weights with the fixed simulation timestep $\Delta t = 0.11$ s. The six thermal capacitance and paste-resistance inverses are pre-computed once as module-level constants to avoid repeated floating-point division inside the integration loop, in direct correspondence with the Python implementation. The `stepPhysics` function implements the identical Forward Euler integration and sigmoid-modulated convection model as `step_physics_numba`, operating on a `ThermalState` object that holds the four state variables $T_{\text{die},0}$, $T_{\text{die},1}$, $T_{\text{sink},0}$, $T_{\text{sink},1}$ for a single node.

B. CSV Ingestion and Job Parsing

The dashboard accepts MIT Supercloud or any custom trace files directly in the browser via `parser.ts`. Parsing is delegated to the PapaParse library with its native `worker: true` flag, which automatically offloads the CSV tokenization to a background thread, keeping the main UI thread free during file ingestion. The parsed rows are split by `gpu_index` into two power trace arrays. A file containing rows for both GPU 0 and GPU 1 produces a dual-GPU job (`requested_gpus: 2`); a file with only one GPU index produces a single-GPU job mapped to `power_trace_0`. The resulting `Job` objects carry the same fields as the Python implementation power traces, work-deficit accumulators, per-GPU throttle step counters, and arrival and start timestamps and are forwarded to the simulation Web Worker for scheduling. A pre-parsed quick-start bundle (`quickstart_bundle.json`) is prefetched into browser memory on application mount so that first-time users can launch a demonstration simulation without uploading any files.

C. Web Worker Simulation Engine

The entire simulation runs in the background using a dedicated Web Worker (`worker.ts`). This architectural choice ensures that the heavy mathematical computations like the ODE physics integration and continuous scheduler logic happens entirely off the main thread, keeping the user interface

smooth and responsive. Rather than relying on the main application to track the simulation, the Web Worker manages the complete `SimulationState` internally and communicates with the UI via a simple messaging system. For example, when the worker receives an `INIT` setup command, it spins up the virtual server nodes, sets them to the requested ambient temperature, and clears out any old queue data. The main application then passes the batch of uploaded job traces to the worker using an `ADD_JOBS` command prior to launching the simulation loop.

Each call to the internal `tick()` function advances the simulation by one timestep. Within a tick, the worker executes the identical logic established by the Python based simulator explained in Section VII.

D. Master Clock and A/B Lockstep Synchronisation

In Standard or Thermal-Aware scheduler mode, the worker drives itself via an internal `setInterval` at the simulation tick rate, executing `stepsPerFrame` ticks per interval according to the user-selected speed multiplier, and emitting a `STATE_DELTA` message after each batch.

When A/B testing is enabled, two independent worker instances are spawned: Worker A is initialised in Standard mode and Worker B in Thermal-Aware mode. Both receive deep copies of the same job queue with identical arrival timestamps. Rather than running on independent internal clocks, both workers are placed in lockstep by a *master clock* running on the main thread as a `requestAnimationFrame` loop. On each animation frame, the master clock accumulates wall-clock elapsed time against a 110 ms or 0.11s tick interval. When sufficient time has accumulated, it sends a `TICK_CHUNK` message carrying the number of ticks to execute to both workers simultaneously, and marks both workers as busy. Neither worker is issued another `TICK_CHUNK` until both have replied with `CHUNK_COMPLETE`, enforcing cycle-exact synchronisation between the two simulated universes. This guarantees that the A/B comparison panel displays both schedulers at the same simulated wall-clock instant at all times.

E. Delta Compression and Rendering Pipeline

To avoid the overhead of serializing and transferring the full simulation state on every tick, the worker maintains internal tracking counters and constructs a `WorkerDelta` message containing only the incremental updates since the previous transmission. The new completed job statistics, chart time labels, and per-node temperature and power arrays are simply appended to the message payload.

Instead of forcing the UI to be updated every time a new piece of data arrives, the main application quietly collects these incremental updates in background memory. Visual updates to the dashboard are strictly synchronized with the browser's natural display refresh rate so that its speed would be independent of the computation rate of the underlying calculation engine. This prevents the browser from freezing even under heavy computational load. Even the real-time telemetry graphs are programmed to trigger a render only when a new batch of data explicitly arrives, conserving rendering resources preventing unnecessary chart re-renders.

F. Data Export

The dashboard supports two export granularities. The sampled export re-uses the downsampled chart data already accumulated in the live buffers (one sample per 50 ticks, approximately 5.5 s intervals) and packages per-node telemetry CSVs together with interactive HTML dashboards into a ZIP archive for download. The high-resolution export reruns the full simulation silently inside the worker under the `RUN_EXPORT` message handler, logging every tick at the full $\Delta t = 0.11$ s resolution. To avoid exhausting browser heap memory when simulating large clusters across long execution times, the export worker first attempts to acquire a `FileSystemSyncAccessHandle` via the Origin Private File System (OPFS) API. If granted, telemetry rows are flushed to a private on-disk file in chunks of 5000 lines using the synchronous `write` method, which is permissible inside a Web Worker context. If OPFS access is blocked by the browser, the implementation falls back to in-memory `Blob` accumulation. Upon completion, the worker transfers the resulting file handles or blobs back to the main thread, which assembles and triggers a ZIP archive download.

IX. Results and Discussion

The evaluation of this work is partitioned into two sequential phases. First, the predictive accuracy of the calibrated Two-Mass ODE is quantified against unseen test dataset. Second, the calibrated ODE is deployed within the cluster simulator to conduct an A/B testing to comparing the Standard and Thermal-Aware scheduling policies across node counts ranging from 2 to 256.

A. ThermalODE Model Calibration and Predictive Accuracy

1) Training Convergence and Learning Dynamics: The Two-Mass ODE was set to train to a maximum of 500 epochs. The best model checkpoint was recorded at epoch 81, at which point the validation RMSE and MAE reached their global minimum of $2.2752\,^{\circ}\mathrm{C}$ and $1.4424\,^{\circ}\mathrm{C}$ respectively. Figure 2 presents the training and validation RMSE trajectories and it exhibits no signs of overfitting. The absence of a widening gap or an upward trend in validation error confirms that the model generalizes well to unseen data rather than memorizing the training samples. The `ReduceLROnPlateau` scheduler halved the learning rate at several points during training, resulting in subsequent improvement of the validation performance before the model ultimately converged.

2) Calibrated Physical Parameters: A critical validation of the physics-informed approach lies not only in predictive accuracy, but in whether the optimized parameter values converge to physically plausible quantities. The final calibrated values, reported in Table IV, satisfy this requirement across all 18 parameters.

The silicon die capacitances ($C_{\text{die},0} = 8.93$, $C_{\text{die},1} = 8.87\,\mathrm{J/^{\circ}C}$) are small as physically expected the silicon die has minimal thermal mass and heats rapidly under load. In contrast, the heatsink masses ($C_{\text{sink},0} = 4713.6$, $C_{\text{sink},1} = 4831.2\,\mathrm{J/^{\circ}C}$) converged near the upper boundary of their search space, consistent with the large heatsink and chassis

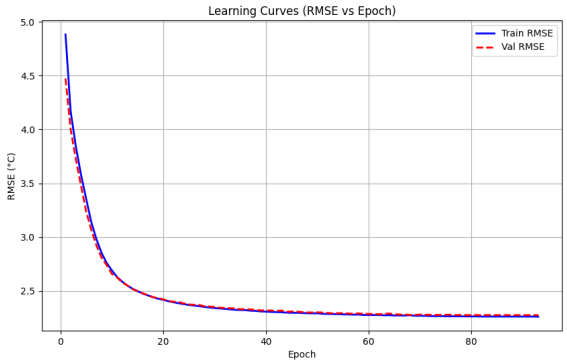

Fig. 2. Training and validation RMSE curve.

structures in a server-class GPU node, which act as significant thermal reservoirs. The paste resistances ($R_{\text{paste},0} = 0.037$, $R_{\text{paste},1} = 0.034\,°\text{C/W}$) are physically consistent with standard high-performance thermal interface materials.

The dynamic fan curve parameters are notably interpretable. The base convective cooling coefficients ($h_{\text{base},0} = 3.98$, $h_{\text{base},1} = 4.76$ W/°C) are appropriately small compared to their active counterparts. These values physically represent the baseline thermal dissipation provided by the server chassis' constant idle airflow before the fans aggressively ramp up, establishing a realistic lower bound for continuous heat removal. Both GPUs calibrated active cooling coefficients as $h_{\text{active},0} = 20.91$ and $h_{\text{active},1} = 19.82$, with fan engagement thresholds at $T_{\text{thresh},0} = 70.25\,°\text{C}$ and $T_{\text{thresh},1} = 67.51\,°\text{C}$. The steepness coefficients ($\beta_0 = 1.669$, $\beta_1 = 1.335$) indicate a sharp fan ramp, reflecting the aggressive step-like behavior of enterprise level GPU hardware.

The crosstalk coefficients reduced significantly during training from their empirically derived initial values. The final calibrated values ($k_{01} = 0.0167$, $k_{10} = 0.0028$) are substantially lower than the prior estimates ($\kappa_{01} = 0.0780$, $\kappa_{10} = 0.0281$). This reduction is physically consistent: the priors were extracted from isolated asymmetric events where one GPU was idle, maximizing observed thermal crosstalk. During normal dual-GPU operation which dominates the training dataset both GPUs are simultaneously active, and the crosstalk signal is largely absorbed into the heatsink's thermal reservoir rather than manifesting as a measurable die temperature rise. The optimizer correctly identified this distinction. Finally, the strongly negative bias values ($q_0 = -8.92$, $q_1 = -8.94\,\text{W}$) are physically interpretable as a corrective term for the underestimation of convective airflow from the server chassis a heat sink effect that is not captured by the two-GPU-centric model formulation.

A striking similarity exists between the calibrated parameter values for GPU 0 and GPU 1. Since these are identical NVIDIA V100 accelerators sitting in the same chassis node, their true physical properties are inherently near-identical. The fact that the gradient descent optimization independently converged on nearly matching physical profiles for both independent GPUs, without being forced to do so, serves as strong evidence that the true underlying thermodynamics of the hardware has been successfully modelled, rather than relying on arbitrary statistical fits.

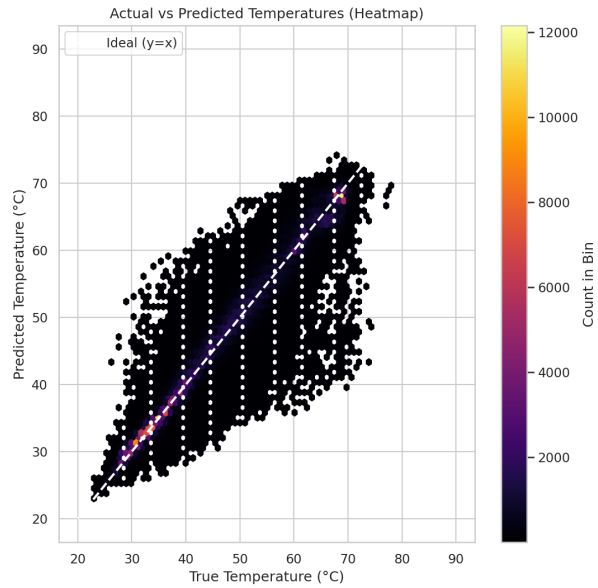

Fig. 3. Hexbin density heatmap of predicted versus true GPU temperatures across a 500,000-point random subsample of over 443 million evaluated states. The high-density ridge along the diagonal ($y = x$) indicates strong global calibration with minimal systematic bias.

3) Global Predictive Performance: To quantify generalization capability, evaluation was executed across a separate hold-out test set comprising 671 distinct physical jobs, evaluating a total of 443,815,380 timesteps representing thousands of hours of real-world datacenter operation.

The ThermalODE achieved a global RMSE of $\mathbf{2.2849\,°C}$ and a global MAE of $\mathbf{1.4561\,°C}$. Figure 3 presents the density heatmap of predicted versus true temperatures, where the concentration of mass along the diagonal confirms strong global calibration with minimal systematic offset.

A directional bias of $+0.1955\,°\text{C}$ was measured, indicating a slight systematic tendency to overestimate rather than underestimate temperature. In the context of predictive thermal scheduling, this conservative bias is operationally preferable as an overestimating simulator will trigger thermal-aware interventions fractionally earlier than strictly necessary, whereas an underestimating model risks missing genuine hardware stress events.

4) Tail-Risk and Extreme State Evaluation: For predictive scheduling, the reliability of an ODE based model at its tail is more operationally critical than its median performance. A model that is accurate on average but prone to large error deviations at thermal extremes cannot be safely used for making informed scheduling decisions. Figure 4 presents the absolute error distribution across the test set.

The model maintained an absolute error below $4.66\,°\text{C}$ for 95% of all predictions, and below $7.95\,°\text{C}$ for 99% of the 443 million evaluated states. The observed maximum error of $34.68\,°\text{C}$ occurs in power-step events where the die tempera-

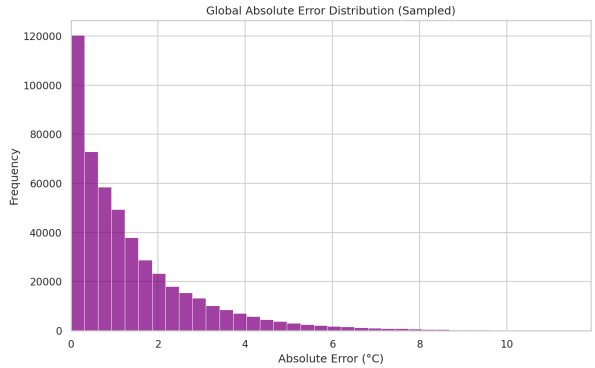

Fig. 4. Absolute error distribution across 500,000 randomly subsampled test timesteps. The distribution is strongly concentrated below 5 °C, with the 95th and 99th percentile boundaries annotated. The long tail corresponds to transient step-response events.

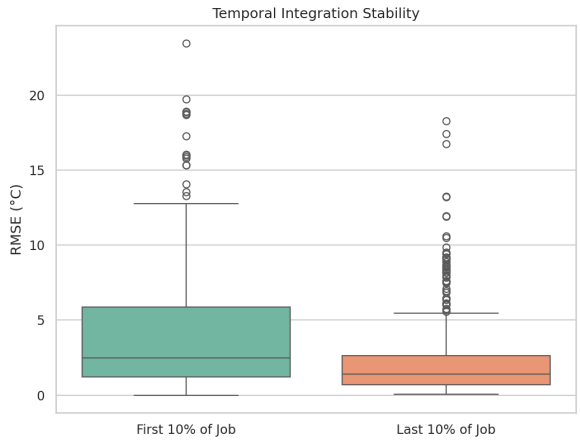

Fig. 5. Per-job RMSE comparison between the first 10% and last 10% of each job's duration, evaluated across all 671 test jobs. The lower median and tighter interquartile range at the tail of jobs demonstrates that integration errors do not amplify over prolonged workloads.

ture changes much faster than the Euler-based integrator can keep up with at a $\Delta t = 0.11$ s timestep, a known limitation of explicit first-order integration under steep gradient conditions.

An important portion of the evaluation focused exclusively on the *Danger Zone*, defined as any timestep where the true hardware temperature exceeded 70.0 °C the same threshold used as the projection rejection gate in the Thermal-Aware scheduler in Section VII. Within this high-stress boundary, the model maintained a RMSE of 5.3853 °C. While this is higher than the global baseline, the physically meaningful point is that the model does not collapse into divergence at the boundary that matters most for scheduling decisions. The sigmoid-gated $h(T_{\text{die}})$ formulation correctly captures the nonlinear fan engagement in this regime, as evidenced by the calibrated T_{thresh} values falling within this danger zone (70.25 °C and 67.51 °C for GPU 0 and GPU 1, respectively).

5) Temporal Integration Stability: A fundamental risk in autoregressive ODE simulation is temporal drift. Prediction errors from early timesteps can accumulate and compound throughout the sequence, causing the model to diverge from physical reality over long workloads. To quantify this, RMSE was computed separately for the first 10% and last 10% of each job's duration, with results shown in Figure 5.

Counterintuitively, the model demonstrates improving accuracy over the course of a workload. The mean RMSE for the first 10% of job execution is 3.7968 °C, which converges to 2.1676 °C during the final 10%. This behavior is physically explained by the initialization assumption: the heatsink temperature at $t = 0$ is not directly observable and is estimated via the thermal equilibrium approximation $T_{\text{sink}}(0) = T_{\text{die}}(0) - P(0) \cdot R_{\text{paste}}$. For jobs that begin with a sudden cold-start power spike, this initialization underestimates the true thermal lag, causing elevated early-stage errors. As the heatsink reaches its true steady-state, the model's predictions converge. The absence of error inflation over prolonged sequences confirms that the physics-informed ODE model successfully constrains the integration within a stable thermodynamic region, unlike unconstrained recurrent networks whose errors diverge on long horizons.

B. A/B Testing: Scheduler Policy Evaluation

1) Experimental Configuration: To evaluate the operational impact of the Thermal-Aware scheduling policy, A/B Testing was conducted on the cluster simulator as explained in Section VII. A cooling degradation of 65% (values of h_{base} and h_{active} were set to 35% of their optimized values from Table IV) was simulated on a homogeneous workload of 56 VGG19 training jobs was submitted to both the Standard (FCFS) and Thermal-Aware schedulers under identical conditions, with a fixed ambient temperature of 30 °C. The cluster size was scaled across eight tiers: 2, 4, 8, 16, 32, 64, 128, and 256 nodes, making node count the sole independent variable. Across all configurations, the per-job execution time remained constant at 45,565.23 s, and no hardware thermal throttling events were observed in either scheduler as the maximum recorded temperature remained below the 87.0 °C throttle threshold in every run. The full results are reported in Table V.

2) Regime Analysis: The results reveal three operationally distinct regimes that are critical to interpreting the tradeoffs of the Thermal-Aware policy.

a) Regime I: Resource-Constrained (2-16 Nodes): Under severe resource contention, the Thermal-Aware scheduler incurs a substantial latency penalty. At 2 nodes, makespan increases from 665,363 s to 847,325 s (+27.4%), and average job wait time increases from 273,855 s to 371,638 s (+35.7%). This overhead is a direct and mechanistically expected consequence of the scheduler's two sequential rejection gates. The projection limit gate (Gate 1) holds jobs in the queue when any candidate node's 5-minute thermal projection exceeds 70.0 °C; the cooldown gate (Gate 2) rejects nodes whose current die temperature exceeds 65.0 °C, even if the projection would pass. At 2 nodes, the Standard scheduler's *Avg Min Temp* of 60.59 °C reveals that GPUs almost never return to idle baseline temperatures between consecutive jobs as the scheduler immediately reassigns hardware from one job to the

TABLE V
A/B TESTING RESULTS: 56 VGG19 WORKLOADS ACROSS VARIABLE NODE COUNTS.

Metric	Scheduler	2	4	8	16	32	64	128	256
Makespan (s)	Standard	665363.82	348939.80	203354.14	120711.14	85726.96	85726.96	85726.96	85726.96
	Thermal-Aware	847325.27	438886.69	245801.16	150883.15	85726.96	85726.96	85726.96	85726.96
Avg Wait Time (s)	Standard	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23
	Thermal-Aware	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23	45565.23
Avg Exec Time (s)	Standard	273855.22	123761.44	49968.09	12901.91	0.00	0.00	0.00	0.00
	Thermal-Aware	273855.22	123761.44	49968.09	12901.91	0.00	0.00	0.00	0.00
Avg Min Temp (°C)	Standard	60.59	58.17	53.43	43.99	30.00	30.00	30.00	30.00
	Thermal-Aware	45.17	44.63	42.06	36.86	30.00	30.00	30.00	30.00
Global Max Temp (°C)	Standard	72.26	72.22	72.28	72.26	72.26	72.26	72.26	72.26
	Thermal-Aware	72.27	72.28	72.18	72.28	72.26	69.71	69.71	69.71
Avg Max Temp (°C)	Standard	70.76	70.62	70.67	70.62	70.71	70.71	70.71	70.71
	Thermal-Aware	70.94	70.89	70.80	70.76	70.53	69.43	69.43	69.43
Avg Mean Temp (°C)	Standard	69.23	69.05	69.04	68.88	68.76	68.76	68.76	68.76
	Thermal-Aware	69.27	69.19	69.05	68.93	68.58	67.50	67.50	67.50
Avg Std Dev (°C)	Standard	1.05	1.24	1.58	2.20	3.24	3.24	3.24	3.24
	Thermal-Aware	1.78	1.89	2.14	2.55	3.22	3.15	3.15	3.15

next. The Thermal-Aware policy enforces cooldown, reducing *Avg Min Temp* to $45.17\,°\mathrm{C}$, confirming that the cooldown gates are functioning as designed. However, with only 2 nodes serving 56 jobs, each enforced cooldown period translates directly into queuing delay.

Critically, even in this constrained regime, the per-job *Avg Exec Time* remains identical at $45{,}565.23\,\mathrm{s}$ in both policies. This confirms that the additional latency is entirely pre-execution queuing overhead, not mid-execution performance degradation. The Thermal-Aware scheduler transfers thermal risk from hardware stress during execution to idle wait time before execution, a tradeoff that is operationally preferable in environments where hardware reliability outweighs throughput.

b) Regime II: Saturation Point (32 Nodes): At 32 nodes, both schedulers converge to identical makespan and zero average wait time. With sufficient nodes to absorb the full workload without queuing, the Thermal-Aware scheduler's rejection gates never activate as there is always a thermally safe candidate available. This saturation point defines the resource threshold above which the Thermal-Aware policy achieves thermal benefits without any latency cost. The *Avg Std Dev* values at this tier ($3.22\,°\mathrm{C}$ vs $3.24\,°\mathrm{C}$) are nearly identical, indicating that both schedulers distribute load similarly when resources are unconstrained.

c) Regime III: Resource-Abundant (≥64 Nodes): Beyond saturation, the Thermal-Aware scheduler's thermal optimization becomes fully decoupled from latency. Both schedulers produce zero wait time and identical makespan across 64, 128, and 256 nodes. The divergence appears entirely in the thermal domain. The Standard scheduler's *Global Max Temp* remains fixed at $72.26\,°\mathrm{C}$ across all node counts from 32 onward, a floor imposed by the workload intensity of VGG19 training. The Thermal-Aware scheduler, by intelligently distributing load away from thermal hotspots, suppresses the *Global Max Temp* to $69.71\,°\mathrm{C}$. For 64 nodes and above a reduction of $2.55\,°\mathrm{C}$ below the Standard policy's peak is observed.

At the mean temperature level, the effect is equally significant. At 64+ nodes, the Thermal-Aware scheduler achieves an *Avg Mean Temp* of $67.50\,°\mathrm{C}$ compared to $68.76\,°\mathrm{C}$ for the Standard policy, a reduction of $1.26\,°\mathrm{C}$ was observer for 64 nodes and above.

3) Physical Significance of Thermal Reduction: The $1.26\,°\mathrm{C}$ mean temperature reduction and the $2.55\,°\mathrm{C}$ peak reduction observed at 64+ nodes may appear numerically modest. Their engineering significance, however, operates through two mechanisms.

First, the Standard scheduler consistently operates within $0.71\,°\mathrm{C}$ of the projection rejection limit ($70.0\,°\mathrm{C}$) across all configurations, indicating that it routinely approaches the thermal boundary at which the Thermal-Aware gate would intervene. Any increase in workload intensity, ambient temperature, or rack density would push the Standard scheduler above this boundary, triggering hardware throttling events. The Thermal-Aware policy maintains a $0.29\,°\mathrm{C}$ or larger margin to this boundary even in the stress regime, representing a structural safety buffer.

Second, the *Avg Min Temp* behavior confirms that the Standard scheduler provides substantially less inter-job cooldown at low node counts. At 16 nodes, the Standard *Avg Min Temp* of $43.99\,°\mathrm{C}$ versus the Thermal-Aware $36.86\,°\mathrm{C}$ reflects a $7.13\,°\mathrm{C}$ lower floor temperature, directly reducing thermal stress on the hardware.

4) Temperature Variance and Load Distribution: The *Avg Std Dev* metric captures the spread of temperature across the various number or nodes and serves as an indicator of thermal load balance. Across all configurations, the Thermal-Aware scheduler maintains a marginally lower standard deviation than the Standard policy at all node counts from 32 onward ($3.15\,°\mathrm{C}$ vs $3.24\,°\mathrm{C}$ at 64-256 nodes). While this difference is small, it confirms that the predictive placement is actively distributing thermal load more uniformly across the cluster rather than creating isolated hotspot nodes, which is the dominant failure mode of FCFS in large heterogeneous HPC deployments.

5) Limitations and Scope: The homogeneous VGG19 workload used in this study represents a controlled, worst-case thermal scenario due to the sustained high power draw characteristic of deep convolutional training. Real HPC workloads exhibit heterogeneous power profiles across jobs, which would modulate the relative advantage of the Thermal-Aware policy. Additionally, the simulation fixes the ambient temperature at $30\,^{\circ}\mathrm{C}$; in practice, datacenter cooling systems themselves respond to thermal load, creating a feedback loop that this simulator does not model.

Furthermore, the per-job execution time was observed to be identical across both schedulers ($45{,}565.23\,\mathrm{s}$), and no throttling events occurred in either policy. This validates that the Thermal-Aware scheduler's intervention is entirely pre-execution. The Thermal-Aware scheduler will gain edge over Standard scheduler when high temperatures and throttling events are observed rather than simple placement and queuing decisions under moderate temperatures.

X. Conclusion and Future Work

Given the increase in GPU dense datacenters, replacing reactive hardware throttling to proactive thermal management is crucial. This paper presented ThermalODE, a physics-informed digital twin and scheduling framework designed to predict and mitigate thermal bottlenecks in HPC clusters.

Due to the strict enforcement of thermodynamic boundaries for parameter optimization, the ThermalODE model achieved exceptional predictive accuracy, demonstrating a global RMSE of $2.28\,^{\circ}\mathrm{C}$ across over 443 million evaluated timesteps. The transition of this ODE model into a JIT compiled cluster simulator enables the testing of various scheduling algorithms and a performance comparison between Standarad FCFS and Thermal-Aware predictive scheduling was demonstrated. Rather than reacting to thermal emergencies, the Thermal-Aware policy leveraged predicted future temperature projections to make informed workload assignments and passive cooldown locks. This approach successfully reduced the peak global temperatures by up to $2.55\,^{\circ}\mathrm{C}$ compared to standard FCFS allocation. By transferring thermal risk from mid-execution hardware stress to pre-execution queue wait times, the scheduler guarantees deterministic execution speeds for active workloads.

Future work will focus on expanding the physics-informed model to encompass the broader ambient datacenter environment. Integrating room-HVAC dynamics and liquid-cooling parameters into the ODE formulation will enable a complete, facility-wide digital twin. A constant worst-case power draw of $P_{\mathrm{proj}} = 250\,\mathrm{W}$ is assumed for future temperature projection in Thermal-Aware scheduling. A better algorithm can be designed to dynamically establish the P_{proj} instead of a static constant. Current model explicitly focuses on GPUs within a cluster, extending the model to incorporate the CPUs will facilitate heterogeneous thermal scheduling across diverse architectures.

Acknowledgments

The authors would like to thank Dr. Vishwanath Karad MIT World Peace University, particularly the Department of Computer Engineering and Technology, for providing the computational resources necessary for this work. Specifically, we acknowledge the use of the PARAM Shavak DL system and its NVIDIA RTX A4500 GPU, which accelerated the computational workloads in this study.

References

[1] A. B. Yoo, M. A. Jette, and M. Grondona, "Slurm: Simple linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing*, D. Feitelson, L. Rudolph, and U. Schwiegelshohn, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 44–60.

[2] NVIDIA Corporation, "NVIDIA Tesla V100 GPU Architecture," NVIDIA, Whitepaper, 2017. [Online]. Available: https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019. [Online]. Available: https://arxiv.org/abs/1912.01703

[4] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: a llvm-based python jit compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, ser. LLVM '15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: https://doi.org/10.1145/2833157.2833162

[5] S. Go, J. Park, S. More, H. Wu, I. Wang, A. Jezghani, T. Krishna, and D. Mahajan, "Characterizing the efficiency of distributed training: A power, performance, and thermal perspective," 2025. [Online]. Available: https://arxiv.org/abs/2509.10371

[6] Y. Lee, K. G. Shin, and H. S. Chwa, "Thermal-aware scheduling for integrated cpus–gpu platforms," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3358235

[7] B. Kocot, P. Czarnul, and J. Proficz, "Energy-aware scheduling for high-performance computing systems: A survey," *Energies*, vol. 16, no. 2, 2023. [Online]. Available: https://www.mdpi.com/1996-1073/16/2/890

[8] Y. Liu, P. S. Shah, T. Xia, and D. Huston, "Self-referencing digital twin for thermal and task management in package stacked esp32-s3 microcontrollers with mixture-of-experts and neural networks," *Computers*, vol. 15, no. 1, 2026. [Online]. Available: https://www.mdpi.com/2073-431X/15/1/4

[9] J. Ran, Q. Zhang, Y. Zhu, J. Zhai, J. Li, Z. Guo, and T. Wang, "Co-optimization of thermal-aware workload scheduling with deep reinforcement learning-based cooling control in data centers," *Energy*, vol. 344, p. 139965, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360544226000678

[10] L. Di Natale, B. Svetozarevic, P. Heer, and C. Jones, "Physically consistent neural networks for building thermal modeling: Theory and analysis," *Applied Energy*, vol. 325, p. 119806, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306261922010819

[11] M. Maiterth, W. H. Brewer, J. S. Kuruvella, A. Dey, T. Z. Islam, R. Kabir, K. Menear, D. Duplyakin, T. Patki, T. Jones, and F. Wang, "Hpc digital twins for evaluating scheduling policies, incentive structures and their impact on power and cooling," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25. ACM, Nov. 2025, p. 1959–1969. [Online]. Available: http://dx.doi.org/10.1145/3731599.3767559

[12] J. Stojkovic, C. Zhang, Íñigo Goiri, E. Choukse, H. Qiu, R. Fonseca, J. Torrellas, and R. Bianchini, "Tapas: Thermal- and power-aware scheduling for llm inference in cloud platforms," 2025. [Online]. Available: https://arxiv.org/abs/2501.02600

[13] Y. Zhang, Y. Kurkure, Y. Tao, M. E. Papka, and Z. Lan, "A real-time digital twin for adaptive scheduling," 2025. [Online]. Available: https://arxiv.org/abs/2512.18894

[14] S. Samsi, M. L. Weiss, D. Bestor, B. Li, M. Jones, A. Reuther, D. Edelman, W. Arcand, C. Byun, J. Holodnack, M. Hubbell, J. Kepner, A. Klein, J. McDonald, A. Michaleas, P. Michaleas, L. Milechin, J. Mullen, C. Yee, B. Price, A. Prout, A. Rosa, A. Vanterpool, L. McEvoy, A. Cheng, D. Tiwari, and V. Gadepally, "The mit supercloud dataset," in *2021 IEEE High Performance Extreme Computing Conference (HPEC)*, 2021, pp. 1–8.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[16] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022